CERTIFIED HYBRID DECODING OF BINARY LDPC CODES OVER THE BEC VIA SUBEXPONENTIAL LINEAR LOGIC (SELL)

CARLOS RAMÍREZ OVALLE

Pontificia Universidad Javeriana – Cali, Colombia
e-mail address: carlosovalle@javerianacali.edu.co

Abstract. We develop a proof-theoretic certification layer for hybrid LDPC decoding over the binary erasure channel (BEC). The decoder has two phases: (i) a peeling phase that resolves variables greedily from degree-one check nodes, and (ii) a residual phase that solves the remaining sparse linear system over $\mathbb{F}_2$ by Gaussian elimination. We specify phase (i) in a clause-based fragment of linear logic with subexponentials (SELL), interpreted as multiset rewriting, and prove that this specification is *adequate*: it induces exactly the same executions as the operational decoder.

From any derivation the specification extracts a compact trace; an independent checker then validates the decoded word against that trace and a residual witness. We prove soundness of success certificates—any accepted output is a codeword consistent with the received non-erased symbols and, under the BEC model, is maximum-likelihood—and we prove both soundness and completeness of failure certificates, which witness infeasibility of the residual system when the BEC assumption is violated. The checker runs in time $O(|\mathcal{E}| + n)$, linear in the Tanner graph.

The four informal results together form a self-contained pipeline: specification, adequacy, certification, and checking. Because the adequacy proof proceeds entirely by structural induction over multiset-rewriting derivations, its inductive skeleton maps directly onto the tactic language of contemporary proof assistants. To substantiate this claim concretely, we provide a companion Lean 4 library of approximately 650 lines covering all type definitions, the rewriting rules, and the well-formedness preservation lemmas. The soundness theorem for success certificates is proved in Lean 4; the ML optimality theorem is proved conditional on a BEC probability axiom that is stated as a genuine axiom rather than a placeholder. The remaining four admitted steps span two soundness items and two adequacy items; each is annotated with the corresponding Mathlib API and a precise line-count estimate for the remaining work. A companion Python implementation and a reproducibility notebook allow the certificate format and checker algorithm to be inspected and executed independently of the proof assistant.

1. Introduction

Error-correcting codes transmit information reliably over noisy channels by adding structured redundancy. Among the most successful families are LDPC codes, whose sparse parity-check structure enables efficient iterative decoding. This paper is concerned not with designing a new decoder, but with certifying that an existing one is correct.

Key words and phrases: LDPC codes, binary erasure channel, peeling decoder, linear logic with subexponentials, multiset rewriting, certified decoding, proof-carrying code, Lean 4 mechanization.

Preprint submitted to
Logical Methods in Computer Science

© C. Ramírez Ovalle
Creative Commons

Let $H \in \mathbb{F}_2^{m\times n}$ be a sparse parity-check matrix and let

$$C = \{x \in \mathbb{F}_2^n \mid Hx^\top = 0\}$$

be the associated binary LDPC code [2, 3]. Over the binary erasure channel (BEC), a received word is $y \in (\mathbb{F}_2 \cup \{\star\})^n$, where $\star$ marks an erased coordinate. The decoding task is to recover a codeword consistent with the non-erased positions of y.

For the BEC, the first decoding phase is classical and particularly transparent: the *peeling decoder* repeatedly resolves a variable whenever some parity-check equation contains exactly one unknown. This procedure is cheap, local, and well adapted to sparse Tanner graphs, but it is only partial—it stops precisely when the residual erasure pattern forms a stopping set [6]. The standard remedy is to solve the remaining sparse linear system over $\mathbb{F}_2$ by structured Gaussian elimination, which corresponds to maximum-likelihood (ML) decoding on the BEC [7].

This paper does not propose a new decoder. Its goal is different: to add a *certification layer* to the standard pipeline. In practical terms, the decoder is allowed to be fast, highly optimized, or even treated as untrusted code. What we require is that its output come with a witness small enough for an independent checker to verify quickly. If the checker accepts, the returned word really is a codeword consistent with the received non-erased symbols; under the BEC model, this makes it an ML completion. If no completion exists—say, because "known" bits were silently corrupted—the decoder may instead return a short witness of infeasibility, which the checker can also validate. We model the decoder throughout as an untrusted oracle: a function $\mathcal{D}$ that receives (H, y) and returns a certificate of either type. No assumption is made on the computational behavior of $\mathcal{D}$; it may be adversarially constructed, buggy, or simply produce certificates that the checker will reject. The checker is the only trusted component.

This perspective connects two bodies of literature that have largely developed in parallel. Coding theory already supplies both algorithmic ingredients: peeling and residual linear solving. Proof theory supplies disciplined ways to specify stateful computations and extract traces from derivations. What seems to be missing is a mathematically precise bridge between the two: a proof-theoretic account of the peeling phase that is tight enough to support rigorous certificates, yet stays close to the algebraic language in which coding theorists formulate the residual solve. That bridge is the role of linear logic with subexponentials (SELL) in this paper.

We want to be clear about how SELL is used. It is not decoration, but neither do we invoke the full metatheory of focused proof systems. What we use is a clause-based fragment whose operational reading is multiset rewriting. The subexponentials are essential because they let us separate three kinds of data that coexist in peeling: the static Tanner graph, persistent knowledge about variables that have been determined, and one-use residual state that must be consumed and updated. That tripartite separation is natural logically, but it is also practically useful: it turns a decoding run into a trace that can be replayed and checked independently of the solver.

A natural question is whether this subexponential structure is genuinely necessary or merely convenient. Consider what happens with plain multiset rewriting (no ! operators). To allow $\mathsf{val}(i, b)$ to be reused by multiple adjacent checks, one would need to either (a) make the atom persistent by convention—reintroducing the semantic content of the $!^K$ zone by other means—or (b) duplicate the atom for each check that needs it, which is semantically incorrect (decoding does not copy information; it distributes it). The SELL formalism

handles (a) automatically and rules out (b) without extra bookkeeping. Similarly, without the $!^S$ zone, the edge relation would be a one-use linear resource, and reading it once in a rule would remove it from the graph. The subexponential discipline is therefore not a superficial choice of syntax; it is the mechanism that makes the encoding correct and prevents a class of spurious rewriting sequences.

Positioning and gap. The novelty here is not the decoding algorithm. Peeling, stopping sets, and residual ML decoding over the BEC are all classical; the references in Section 10 trace the relevant history carefully. The gap is more specific: we are not aware of any treatment that simultaneously provides

(a) a SELL-style specification of peeling as explicit multiset rewriting;
(b) a complete adequacy proof connecting that specification with the operational decoder;
(c) explicit success certificates for decoded words *and* explicit failure certificates for residual infeasibility; and
(d) a small checker whose correctness guarantees are stated and proved directly in the language of coding theory.

One could argue that each item individually has appeared somewhere in the literature, in different forms and with different emphases. The point is that having all four together enables something that having them separately does not: a complete pipeline from specification to certification, where the spec is tight enough to justify the checker, and the checker is concrete enough to be implemented and tested.

Contributions.

(i) We define a certified hybrid decoder for binary LDPC codes on the BEC: peeling in phase I, residual Gaussian elimination over $\mathbb{F}_2$ in phase II.
(ii) We introduce two certificate types—a *success certificate* (T, ρ) pairing a peeling trace with a residual assignment, and a *failure certificate* (T, ω) pairing a peeling trace with an infeasibility witness—and give a unified checker for both.
(iii) We give a SELL-style specification of phase I in which subexponentials separate static graph structure, persistent decoded values, and one-use residual state, and we work throughout with the induced multiset-rewriting system.
(iv) We prove the four main theorems: soundness of success certificates (Theorem 5.1), soundness and completeness of infeasibility witnesses (Lemmas 5.3 and 5.4), ML optimality on the clean BEC (Theorem 5.8), and a full adequacy theorem for the peeling phase (Theorem 6.11).
(v) We provide a partial Lean 4 mechanization (Section 8) comprising approximately 650 lines organized in six modules. All type definitions, the rewriting rules, and the well-formedness preservation lemmas are fully proved; the soundness theorems for both certificate types are complete modulo two small admitted helpers (each ≈30 lines of boilerplate). The adequacy theorem has a complete proof skeleton with two admitted steps that are annotated with exact Mathlib API pointers and line-count estimates. We also release a companion Python implementation so that the checker can be executed and inspected independently.

Structure of the paper. Section 2 fixes notation and recalls coding-theoretic prerequisites. Section 3 formulates phase I and proves the stopping-set characterization. Section 4 defines the hybrid decoder and both certificate types. Section 5 proves the main semantic guarantees. Section 6 gives the SELL-style specification and the adequacy theorem. Section 7 discusses robustness beyond the ideal BEC model. Section 8 reports the partial Lean 4 mechanization. Sections 9 and 10 cover the companion implementation and related work.

2. Preliminaries

2.1. **LDPC codes and Tanner graphs.** Let $\mathsf{Var} = \{1, \dots, n\}$ and $\mathsf{Chk} = \{1, \dots, m\}$. The Tanner graph of H is bipartite with edges (j, i) iff $H_{j,i} = 1$ [3, 5]. Write $\mathcal{N}(j) = \{\, i \in \mathsf{Var} \mid H_{j,i} = 1 \,\}$.

For a reader coming from logic rather than coding theory, it is useful to keep the following interpretation in mind. The variable node $i \in \mathsf{Var}$ represents the i-th coordinate of a codeword, while the check node $j \in \mathsf{Chk}$ represents the j-th parity equation. The edge (j, i) simply says that the variable x_i occurs in that equation. Since we work over $\mathbb{F}_2$, the equation attached to a check j is the XOR-parity condition

$$\bigoplus_{i \in \mathcal{N}(j)} x_i = 0.$$

Thus a Tanner graph is a bipartite way of displaying which variables participate in which parity constraints. The adjective "low-density" means that H is sparse, so each variable and each check touches only a small number of neighbors. That sparsity is exactly what makes local decoding procedures such as peeling effective.

2.2. **Binary erasure channel.** A received word is $y \in (\mathbb{F}_2 \cup \{\star\})^n$. Under the ideal BEC model, non-erased symbols are received without error; the channel hides some coordinates but does not corrupt those it reveals. This is the cleanest setting in which the logical and algebraic parts of decoding can be cleanly separated: a known symbol is trustworthy, while an erased symbol is simply an unknown to be recovered from parity constraints.

2.3. **Erasure decoding as a linear system.** Partition indices into known positions $K_y = \{i \mid y_i \in \mathbb{F}_2\}$ and erased positions $B_y = \{i \mid y_i = \star\}$. Any completion $x \in \mathbb{F}_2^n$ consistent with y must satisfy:

$$Hx^\top = 0 \quad \text{and} \quad x_i = y_i \text{ for } i \in K_y.$$

Writing $x = (x_{K_y}, x_{B_y})$ yields the residual system over $\mathbb{F}_2$:

$$H_{B_y} x_{B_y}^\top \;=\; H_{K_y} x_{K_y}^\top. \tag{2.1}$$

This is the algebraic core of erasure decoding: once the known coordinates have been moved to the right-hand side, one only has to solve for the erased coordinates. The hybrid structure studied later mirrors this decomposition. Phase I (peeling) exploits the graph sparsity and resolves variables whenever some parity equation contains exactly one unknown; phase II solves the residual linear system when locality alone is no longer enough.

Example 2.1 (A tiny residual-system example)**.** Consider a code with parity-check matrix $H = \begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \end{pmatrix}$, so the two parity equations are $x_1 \oplus x_2 = 0$ and $x_2 \oplus x_3 = 0$. Suppose the received word is $y = (0, \star, \star)$: variable 1 is known ($y_1 = 0$) and variables $2, 3$ are erased. Substituting $x_1 = 0$ into the first equation leaves $x_2 = 0$; the residual right-hand side is $s_0(1) = 0 \oplus 0 = 0$, and the residual degree is $d_0(1) = 1$. Hence peeling resolves $x_2 = 0$ immediately from check 1. After this step, the second equation $x_2 \oplus x_3 = 0$ also becomes degree-one with residual RHS $s(2) = 0$, so peeling resolves $x_3 = 0$ as well. The resulting trace is $T = [(1, 2, 0), (2, 3, 0)]$, and the success certificate is $(T, \rho = \emptyset)$ since no variables remain. This is the basic pattern that the whole paper formalizes and certifies.

3. Phase I: peeling decoding and stopping sets

3.1. State.

Definition 3.1 (Peeling state)**.** A peeling state is $\sigma = (K, U, s, d)$ where

- $K : \mathsf{Var} \rightharpoonup \mathbb{F}_2$ is a partial assignment (known variables);
- $U = \mathsf{Var} \setminus \mathrm{dom}(K)$ is the set of unknown variables;
- $s : \mathsf{Chk} \to \mathbb{F}_2$ is the residual right-hand-side (XOR of known neighbors);
- $d : \mathsf{Chk} \to \mathbb{N}$ is the residual degree $d(j) = |\mathcal{N}(j) \cap U|$.

Definition 3.2 (Initialization from y)**.** Let $K_0(i) = y_i$ if $y_i \in \mathbb{F}_2$ and undefined otherwise; $U_0 = \mathsf{Var} \setminus \mathrm{dom}(K_0)$. For each check j define

$$s_0(j) = \bigoplus_{i \in \mathcal{N}(j) \cap \mathrm{dom}(K_0)} K_0(i), \qquad d_0(j) = |\mathcal{N}(j) \cap U_0|.$$

Let $\sigma_0 = (K_0, U_0, s_0, d_0)$.

3.2. Stopping sets.

Before giving the formal definition, it is helpful to isolate the intuition. Peeling succeeds only when some parity equation exposes a *single* unknown variable. A stopping set is precisely a pattern of erased variables that blocks this mechanism: every check touching the pattern still sees at least two unknowns, so no local step can start. In that sense, stopping sets are the combinatorial obstruction to phase I.

Definition 3.3 (Stopping set [6])**.** A set $S \subseteq \mathsf{Var}$ is a stopping set if every check adjacent to S has at least two neighbors in S:

$$\forall j \in \mathsf{Chk}\ \big(\mathcal{N}(j) \cap S \neq \emptyset \Rightarrow |\mathcal{N}(j) \cap S| \geq 2\big).$$

Theorem 3.4 (Peeling stalls iff a stopping set remains)**.** *Let $\sigma = (K, U, s, d)$ be a state reachable from σ_0 by valid peeling steps. Peeling stalls in σ (i.e. $U \neq \emptyset$ and no check j has $d(j) = 1$) if and only if U is a nonempty stopping set. In particular, peeling starting from σ_0 fully decodes if and only if the initial erasure set U_0 contains no nonempty stopping set.*

Proof. **Stalling implies stopping set.** Suppose $U \neq \emptyset$ and $d(j) \neq 1$ for every j. Take any check j adjacent to U, meaning $\mathcal{N}(j) \cap U \neq \emptyset$. Then $d(j) = |\mathcal{N}(j) \cap U| \geq 1$; since $d(j) \neq 1$, we get $d(j) \geq 2$. Hence every check adjacent to U has at least two neighbors in U, so U is a stopping set by definition.

Stopping set implies stalling. Conversely, suppose U is a nonempty stopping set. Any peeling step would have to find some check j with $d(j) = 1$, i.e. exactly one neighbor in U. But that would mean j is adjacent to U with $|\mathcal{N}(j) \cap U| = 1$, contradicting the stopping-set condition. Hence no peeling step applies, and the procedure stalls with $U \neq \emptyset$.

Full decoding characterization. Peeling fails to fully decode if and only if it terminates at some reachable state with $U \neq \emptyset$, which by the first part is equivalent to the existence of a nonempty stopping set at that state. Since the unknown set can only shrink during peeling, if U_0 contains no nonempty stopping set then no reachable state has a nonempty stopping set (because any nonempty stopping set reachable later would also be a stopping set contained in U_0, a contradiction). Conversely, if peeling stalls with residual $U \neq \emptyset$, then $U \subseteq U_0$ is a nonempty stopping set in U_0. Thus peeling fully decodes if and only if U_0 contains no nonempty stopping set. □

4. Hybrid decoder, certificates, and checking

4.1. **One peeling step.** A triple (j, i, b) is a valid peeling step in state $\sigma = (K, U, s, d)$ if:

$$d(j) = 1, \quad i \text{ is the unique element of } \mathcal{N}(j) \cap U, \quad b = s(j).$$

Applying it yields $\sigma' = (K', U', s', d')$:

$$K' = K \cup \{i \mapsto b\}, \quad U' = U \setminus \{i\},$$

and for each check ℓ:

$$s'(\ell) = \begin{cases} s(\ell) \oplus b & \text{if } i \in \mathcal{N}(\ell), \\ s(\ell) & \text{otherwise,} \end{cases} \qquad d'(\ell) = \begin{cases} d(\ell) - 1 & \text{if } i \in \mathcal{N}(\ell), \\ d(\ell) & \text{otherwise.} \end{cases}$$

4.2. **Certificates.**

Definition 4.1 (Phase-I trace)**.** A peeling trace is a finite list

$$T = [(j_1, i_1, b_1), \ldots, (j_t, i_t, b_t)],$$

intended to be replayed from y by the checker.

Definition 4.2 (Success certificate)**.** A *success certificate* is a pair (T, ρ) where T is a peeling trace and $\rho : U_t \to \mathbb{F}_2$ assigns values to the unknown set U_t remaining after replaying T.

Definition 4.3 (Failure certificate)**.** A *failure certificate* is a pair (T, ω) where T is a peeling trace and $\omega \in \mathbb{F}_2^m$ is an infeasibility witness for the residual system after replaying T.

4.3. **Hybrid decoder.** The hybrid decoder is the computational core of the proposal. It is allowed to use any efficient linear-algebra routine in phase II; the correctness burden is shifted to the certificate and to the checker. This is why the algorithm below is best read as an *untrusted producer* of either a solution witness or an infeasibility witness.

Algorithm 1 Hybrid decoding: peeling + residual GE (returns success or failure certificate)

Require: Tanner graph $(\mathsf{Var}, \mathsf{Chk}, \mathcal{E})$, received $y \in (\mathbb{F}_2 \cup \{\star\})^n$
Initialize $\sigma_0 = (K_0, U_0, s_0, d_0)$
$T \leftarrow [\,]$; $\sigma \leftarrow \sigma_0$
while $U \neq \emptyset$ and $\exists j \in \mathsf{Chk} : d(j) = 1$ **do**
    pick any j with $d(j) = 1$
    let i be the unique element of $\mathcal{N}(j) \cap U$
    $b \leftarrow s(j)$
    append (j, i, b) to T
    update $\sigma \leftarrow \sigma'$ using (j, i, b)
end while
if $U = \emptyset$ **then**
    return $\mathsf{success}(x, T, \rho = \emptyset)$ where x is induced by K
else
    Build residual system $H_U x_U^\top = r$ from the current state (Def. 4.4)
    Solve it over $\mathbb{F}_2$:
      either obtain some solution $\rho : U \to \mathbb{F}_2$, or obtain an infeasibility witness $\omega \in \mathbb{F}_2^m$
    if a solution ρ is obtained **then**
        construct x by K on known coords and by ρ on U
        return $\mathsf{success}(x, T, \rho)$
    else
        return $\mathsf{failure}(T, \omega)$
    end if
end if

4.4. **Independent checker.** Rather than attempting to decode, the checker confines itself to verifying the evidence supplied by the untrusted solver. Concretely, it replays the peeling trace from the received word y, reconstructs the residual system that the trace determines, and then validates whichever certificate was produced—success or failure. By design, both outcomes require the same replay infrastructure, so the checker is a single, unified algorithm with a branching verification step at the end.

Algorithm 2 Checker for success / failure certificates

Require: Tanner graph $(\mathsf{Var}, \mathsf{Chk}, \mathcal{E})$, received y, certificate of the form (T, ρ) or (T, ω)
Initialize $\sigma_0 = (K_0, U_0, s_0, d_0)$; set $\sigma \leftarrow \sigma_0$
for each triple (j, i, b) in T **do**
    Check $d(j) = 1$, i is unique in $\mathcal{N}(j) \cap U$, and $b = s(j)$
    Update $\sigma \leftarrow \sigma'$ using (j, i, b)
end for
Let current state be $\sigma = (K, U, s, d)$
Build H_U and r from σ (Def. 4.4)
if certificate is (T, ρ) **then**
    Check $\mathrm{dom}(\rho) = U$
    for each check $j \in \mathsf{Chk}$ **do**
        Compute $L_j := \bigoplus_{i \in \mathcal{N}(j) \cap U} \rho(i)$
        Check $L_j = s(j)$
    end for
    Construct $x \in \mathbb{F}_2^n$ by $x_i = K(i)$ if defined, else $x_i = \rho(i)$
    Check channel consistency: for all i with $y_i \in \mathbb{F}_2$, verify $x_i = y_i$
    return accept and output x
else ▷ certificate is (T, ω)
    Check $\omega^\top H_U = 0$ (row vector times matrix in $\mathbb{F}_2$)
    Check $\omega^\top r = 1$
    return accept_unsat
end if

The checker above ends by reconstructing exactly the algebraic object that phase II needs. The next subsection makes this residual system explicit and shows how it is read off from the replayed state.

4.5. **Extracting the residual linear system from a stalled state.** Let $\sigma = (K, U, s, d)$ be the state obtained after replaying the peeling trace T.

For each check $j \in \mathsf{Chk}$, define the residual parity equation

$$\bigoplus_{i \in \mathcal{N}(j) \cap U} x_i \;=\; s(j). \tag{4.1}$$

These m equations form a linear system over $\mathbb{F}_2$ in the unknowns $x_U := (x_i)_{i \in U}$.

Definition 4.4 (Residual matrix from the state)**.** Define $H_U \in \mathbb{F}_2^{m \times |U|}$ by restricting the columns of H to indices in U. Let $r \in \mathbb{F}_2^m$ be the vector with coordinates $r_j := s(j)$. Then (4.1) is equivalent to

$$H_U x_U^\top \;=\; r. \tag{4.2}$$

Lemma 4.5 (Meaning of the state components after replay)**.** *Let $y \in (\mathbb{F}_2 \cup \{\star\})^n$ be a received word and let $\sigma = (K, U, s, d)$ be the state obtained after replaying a valid peeling trace from y. Then, for every check $j \in \mathsf{Chk}$,*

$$d(j) = |\mathcal{N}(j) \cap U| \qquad \textit{and} \qquad s(j) = \bigoplus_{i \in \mathcal{N}(j) \cap \mathrm{dom}(K)} K(i).$$

In words, $d(j)$ is the number of still-unknown neighbors of j, and $s(j)$ is the XOR of the neighbors of j that have already been fixed.

Proof. We argue by induction on the length of the replayed trace. For the empty trace the claim is exactly the initialization in Definition 3.2. Assume it holds for a state $\sigma = (K, U, s, d)$ and apply one valid peeling step (j_0, i_0, b). The new state is $\sigma' = (K', U', s', d')$ with $K' = K \cup \{i_0 \mapsto b\}$ and $U' = U \setminus \{i_0\}$. Fix any check j. If $i_0 \notin \mathcal{N}(j)$, then $d'(j) = d(j)$ and $s'(j) = s(j)$ by definition of the update, while the sets of known and unknown neighbors of j do not change. Hence the required equalities remain true for j. If $i_0 \in \mathcal{N}(j)$, then one unknown neighbor is removed, so

$$d'(j) = d(j) - 1 = |\mathcal{N}(j) \cap U| - 1 = |\mathcal{N}(j) \cap U'|.$$

Moreover i_0 moves from the unknown set to the known set with value b, so by the induction hypothesis

$$s'(j) = s(j) \oplus b = \left(\bigoplus_{i \in \mathcal{N}(j) \cap \mathrm{dom}(K)} K(i) \right) \oplus b = \bigoplus_{i \in \mathcal{N}(j) \cap \mathrm{dom}(K')} K'(i).$$

Thus the claim also holds after the step. The induction is complete. □

Lemma 4.6 (State residual equals the standard algebraic residual)**.** *Let $y \in (\mathbb{F}_2 \cup \{\star\})^n$ be a received word and let $\sigma = (K, U, s, d)$ be the state obtained after replaying a valid peeling trace. Let x_K be the partial assignment given by K on* $\mathrm{dom}(K)$*. Then the residual system* (4.2) *is equivalent to the standard restriction*

$$H_U x_U^\top \;=\; H_K x_K^\top$$

obtained by splitting $x = (x_K, x_U)$ in $Hx^\top = 0$ (cf. (2.1)*).*

Proof. Fix a check (row) j. The parity constraint is

$$\bigoplus_{i \in \mathcal{N}(j) \cap \mathrm{dom}(K)} x_i \;\oplus \bigoplus_{i \in \mathcal{N}(j) \cap U} x_i \;=\; 0.$$

Rearranging over $\mathbb{F}_2$ gives

$$\bigoplus_{i \in \mathcal{N}(j) \cap U} x_i \;=\; \bigoplus_{i \in \mathcal{N}(j) \cap \mathrm{dom}(K)} x_i.$$

By Lemma 4.5,

$$s(j) = \bigoplus_{i \in \mathcal{N}(j) \cap \mathrm{dom}(K)} K(i).$$

Thus the right-hand sides match row-wise, yielding (4.1), hence (4.2). The equality with $H_K x_K^\top$ is the same statement in matrix form. □

5. Main guarantees

5.1. Soundness of success certificates.

Theorem 5.1 (Soundness of success certificates)**.** *If Algorithm 2 accepts a success certificate (T, ρ) for (H, y) and outputs x, then:*

(a) *$x_i = y_i$ for all i with $y_i \in \mathbb{F}_2$ (channel consistency);*
(b) *$Hx^\top = 0$ (thus $x \in C$).*

Proof. Channel consistency is checked explicitly.

For parity correctness, fix any check j and split neighbors into those fixed in $\mathrm{dom}(K)$ and those in U. After replaying T, the state component $s(j)$ equals $\bigoplus_{i \in \mathcal{N}(j) \cap \mathrm{dom}(K)} K(i)$ by Lemma 4.5. The checker enforces $\bigoplus_{i \in \mathcal{N}(j) \cap U} \rho(i) = s(j)$, hence

$$\bigoplus_{i \in \mathcal{N}(j)} x_i = \left(\bigoplus_{i \in \mathcal{N}(j) \cap \mathrm{dom}(K)} K(i) \right) \oplus \left(\bigoplus_{i \in \mathcal{N}(j) \cap U} \rho(i) \right) = s(j) \oplus s(j) = 0.$$

Thus every parity equation holds and $Hx^\top = 0$. □

Remark 5.2 (Brevity of the proof is the point)**.** The proof of Theorem 5.1 is short because the checker is designed to be directly verifiable: it makes the correctness argument one-step. This brevity is the primary technical virtue of the certification approach, not a sign of triviality. If the checker were complex—say, a full decoder operating independently—its own correctness would require a separate proof of comparable difficulty, defeating the purpose of separating the untrusted solver from the trusted kernel.

5.2. Soundness of failure certificates (infeasibility witnesses).

We formalize what it means for a residual system to be inconsistent and how to certify this.

Lemma 5.3 (Infeasibility witness)**.** *Let $A \in \mathbb{F}_2^{m \times t}$ and $b \in \mathbb{F}_2^m$. If there exists $\omega \in \mathbb{F}_2^m$ such that*

$$\omega^\top A = 0 \quad \text{and} \quad \omega^\top b = 1, \tag{5.1}$$

then the linear system $Ax = b$ has no solution in $\mathbb{F}_2^t$.

Proof. Assume for contradiction that $x \in \mathbb{F}_2^t$ satisfies $Ax = b$. Left-multiplying by $\omega^\top$ yields

$$\omega^\top A x = \omega^\top b.$$

The left-hand side is $(\omega^\top A)x = 0 \cdot x = 0$, while the right-hand side is 1 by (5.1). This contradiction shows no such x exists. □

Lemma 5.4 (Completeness of infeasibility witnesses)**.** *Let $A \in \mathbb{F}_2^{m \times t}$ and $b \in \mathbb{F}_2^m$. If the system $Ax = b$ has no solution, then there exists $\omega \in \mathbb{F}_2^m$ satisfying* (5.1)*.*

Proof. Let $\mathcal{C} = \mathrm{Col}(A) \subseteq \mathbb{F}_2^m$ be the column space of A. The system $Ax = b$ is solvable iff $b \in \mathcal{C}$. By assumption, $b \notin \mathcal{C}$.

Let $\mathcal{C}^\perp = \{\omega \in \mathbb{F}_2^m \mid \omega^\top c = 0 \ \forall c \in \mathcal{C}\}$ be the orthogonal complement. It is a subspace of $\mathbb{F}_2^m$. Since $\mathcal{C}$ is a subspace and $b \notin \mathcal{C}$, we claim there exists $\omega \in \mathcal{C}^\perp$ such that $\omega^\top b = 1$.

To prove the claim, consider the linear map

$$\phi : \mathcal{C}^\perp \to \mathbb{F}_2, \qquad \phi(\omega) = \omega^\top b.$$

If ϕ were the zero map, then $\omega^\top b = 0$ for all $\omega \in \mathcal{C}^\perp$. But then $b \in (\mathcal{C}^\perp)^\perp$. Over finite-dimensional vector spaces over a field, $(\mathcal{C}^\perp)^\perp = \mathcal{C}$. Hence $b \in \mathcal{C}$, contradicting $b \notin \mathcal{C}$. Therefore ϕ is not the zero map, so there exists $\omega \in \mathcal{C}^\perp$ with $\phi(\omega) = 1$, i.e. $\omega^\top b = 1$.

Finally, $\omega \in \mathcal{C}^\perp$ implies $\omega^\top c = 0$ for every column c of A. Equivalently $\omega^\top A = 0$. Thus (5.1) holds. □

Proposition 5.5 (Witness extraction from row reduction)**.** *Let $A \in \mathbb{F}_2^{m\times t}$ and $b \in \mathbb{F}_2^m$. Suppose a sequence of elementary row operations over $\mathbb{F}_2$ is represented by an invertible matrix $U \in \mathbb{F}_2^{m\times m}$ and transforms the augmented matrix $[A \mid b]$ into*

$$U[A \mid b] = [R \mid c].$$

If the r-th row of R is the zero row and $c_r = 1$, then the row vector

$$\omega^{\mathrm{row}} := e_r^\top U$$

satisfies $\omega^{\mathrm{row}} A = 0$ and $\omega^{\mathrm{row}} b = 1$.

Proof. By definition of U, the r-th row of $[R \mid c]$ is obtained by applying the row combination $e_r^\top U$ to the original augmented matrix. Equivalently,

$$(e_r^\top U)[A \mid b] = [R_{r,*} \mid c_r].$$

Writing $\omega^{\mathrm{row}} := e_r^\top U$, this gives

$$\omega^{\mathrm{row}} A = R_{r,*} \qquad \text{and} \qquad \omega^{\mathrm{row}} b = c_r.$$

Since the r-th row of R is zero and $c_r = 1$, we obtain $\omega^{\mathrm{row}} A = 0$ and $\omega^{\mathrm{row}} b = 1$. □

Remark 5.6. Proposition 5.5 is the algebraic reason why an implementation of phase II can return a concrete failure witness rather than merely reporting that Gaussian elimination found a contradiction. The checker never needs to trust the row-reduction routine itself; it only needs to verify the resulting vector ω^{row}.

Theorem 5.7 (Soundness of failure certificates)**.** *Let (H, y) be an instance and let the checker accept a failure certificate (T, ω). Let $\sigma = (K, U, s, d)$ be the state obtained after replaying T from y, and let $H_U x_U^\top = r$ be the induced residual system. Then there is* no *completion $x \in \mathbb{F}_2^n$ such that $Hx^\top = 0$ and $x_i = y_i$ for all non-erased positions i.*

Proof. Because the checker accepts (T, ω), it verified $\omega^\top H_U = 0$ and $\omega^\top r = 1$. By Lemma 5.3, the residual system $H_U x_U^\top = r$ has no solution. By Lemma 4.6, solvability of the residual system is equivalent to existence of a completion consistent with the fixed coordinates and satisfying $Hx^\top = 0$. Hence no such completion exists. □

5.3. **ML optimality on the BEC.**

Theorem 5.8 (Accepted completions are ML on the BEC)**.** *Assume the ideal BEC model: each coordinate is independently erased with probability ϵ and non-erased coordinates are received without error. Any $x \in C$ consistent with the non-erased positions of y is a maximum-likelihood (ML) solution. In particular, any output accepted by the success checker is ML.*

Proof. For the ideal BEC with parameter $\epsilon \in (0,1)$, the channel transition probability for a received word y given a transmitted codeword x is

$$P(y \mid x) = \begin{cases} \epsilon^{|B_y|}(1-\epsilon)^{|K_y|} & \text{if } x_i = y_i \text{ for all } i \in K_y, \\ 0 & \text{otherwise,} \end{cases}$$

where K_y and B_y are the known and erased index sets of y. This probability depends only on the erasure pattern, not on the particular codeword x—provided x is consistent with the received symbols. Hence every codeword $x \in C$ that agrees with y on K_y achieves the same positive likelihood; all other codewords achieve likelihood zero. The ML criterion therefore accepts any consistent codeword.

By Theorem 5.1, any output accepted by the success checker is (i) a codeword in C and (ii) consistent with all non-erased positions of y. It is therefore ML. □

Remark 5.9 (ML does not imply uniqueness on the BEC)**.** The statement of Theorem 5.8 is deliberately modest. On the BEC there may be several codewords consistent with the non-erased coordinates, and all of them have the same likelihood. The theorem therefore asserts membership in the ML set, not uniqueness of the decoded word.

Remark 5.10 (Why state an essentially immediate result)**.** For the ideal BEC, the equal-likelihood of all consistent codewords makes Theorem 5.8 nearly immediate: any consistent codeword is ML by definition. The value of the theorem is not its depth but its role in the pipeline—it makes explicit that the accepted output of the checker is not merely a valid codeword but an ML solution, without any additional assumption on the decoder. Removing it would leave a gap between the soundness guarantee ("outputs a codeword") and the standard performance criterion ("outputs an ML solution") that a reader might otherwise have to fill in themselves.

Remark 5.11 (Scope: BEC versus other channels)**.** The equal-likelihood argument used in the proof of Theorem 5.8 is specific to the BEC. On the binary symmetric channel (BSC), for example, the likelihood of a codeword depends on its Hamming distance from y, so two different consistent completions can have different likelihoods, and the ML criterion selects the closest one. In that setting, certifying ML optimality would require the certificate to carry distance information, not merely a consistency witness. The present framework does not cover the BSC or AWGN channels; this is a genuine limitation rather than an oversight.

5.4. **Checker complexity (why certification is useful).**

Proposition 5.12 (Checker runs in time linear in the Tanner graph size)**.** *Let $|\mathcal{E}|$ be the number of Tanner edges (equivalently, the number of nonzero entries of H). The success checker runs in $O(|\mathcal{E}| + n)$ time. The failure checker runs in $O(|\mathcal{E}| + m)$ time. Both bounds are tight up to constant factors.*

Proof. **Success case.** Initializing the state from y requires reading all n coordinates and summing, for each check j, the values of its known neighbors; this costs $O(|\mathcal{E}|)$ because the total number of (check, variable) pairs is $|\mathcal{E}|$. Replaying the trace requires, for each step (j, i, b), updating $s(\ell)$ and $d(\ell)$ for every check ℓ adjacent to i: the total work across all t trace steps is at most $\sum_{i \text{ resolved}} \deg(i) \le |\mathcal{E}|$, so $O(|\mathcal{E}|)$ overall. The residual verification loops over all edges (j, ℓ) with $\ell \in U$ and checks $\bigoplus_{\ell \in \mathcal{N}(j) \cap U} \rho(\ell) = s(j)$ for each j; again $O(|\mathcal{E}|)$. The channel-consistency check reads n positions once, costing $O(n)$.

Failure case. Computing $\omega^\top r$ requires one inner product in $\mathbb{F}_2^m$, costing $O(m)$. Checking $\omega^\top H_U = 0$ is equivalent to checking that $\sum_j \omega_j (H_U)_{j,\ell} = 0$ for each column ℓ of H_U. With a sparse representation this sums over at most $|\mathcal{E}|$ nonzeros in total, so the cost is $O(|\mathcal{E}|)$.

Tightness. For the success checker, consider a code where peeling resolves all n variables. The trace has $|T| = n$ entries. Reading each entry (j, i, b) requires updating all checks adjacent to i; summing over all resolved variables, the total work is $\sum_{i=1}^{n} \deg(i) = |\mathcal{E}|$. Any algorithm that verifies every trace entry must perform at least this work, giving a lower bound of $\Omega(|\mathcal{E}| + n)$. For the failure checker, verifying $\omega^\top H_U = 0$ for a dense ω and a sparse H_U requires reading all nonzeros of H_U, costing $\Omega(|\mathcal{E}|)$; reading r costs $\Omega(m)$. Hence both bounds are tight. □

Remark 5.13 (Certificate size)**.** A success certificate (T, ρ) has size $O(n \log n)$ bits: the trace lists $t \le n$ triples from $[m] \times [n] \times \mathbb{F}_2$, each requiring $O(\log n)$ bits to encode. The residual witness ρ contributes at most $|U| \le n$ bits. A failure certificate (T, ω) has the same trace component plus an m-bit witness vector. In both cases the certificate size is linear in the code length up to a logarithmic factor, which matches the $O(n)$ output size of any reasonable decoder and is consistent with the linear-time checker.

6. Clause-based SELL-style specification of phase I (peeling)

This section isolates the part of the construction that genuinely depends on proof theory. The residual linear solve from phase II remains entirely algebraic and is certified afterwards by the witnesses ρ or ω. The only purpose of SELL here is to describe the peeling phase in a way that is precise enough to support a trace and, from that trace, a small independent checker.

Because many readers of this paper will come from coding theory rather than logic, we use only a minimal fragment of the SELL machinery. Nothing in the sequel requires prior familiarity with proof theory beyond the following informal reading. More precisely, we do not invoke the full metatheory of focused SELL in order to establish our main results. What we use is a clause-based fragment whose operational reading is multiset rewriting. The adequacy theorem proved later is therefore an adequacy theorem at the rewriting level, which is exactly the level needed for certification.

6.1. Why SELL appears here.

Peeling updates three qualitatively different kinds of information at each step:

- the Tanner graph edges, which are fixed for the lifetime of the decoding run;
- values assigned to variables that have been decoded, which can be reused whenever another adjacent check needs them; and
- the residual syndrome state of each check node, which must be updated exactly once per resolved neighbor.

Linear logic with subexponentials is useful here because it provides syntactic markers for exactly these three access patterns in a single, uniform calculus. A formula prefixed with a bang, such as $!^S\mathsf{edge}(j, i)$ or $!^K\mathsf{val}(i, b)$, is stored in an unbounded zone: it may be consulted repeatedly without being consumed. A formula with no bang lives in a linear zone and behaves like a one-shot resource: using it in a rule fires once and removes it from the context.

For our application the reading is concrete. The atom $!^S\mathsf{edge}(j,i)$ simply records that (j,i) is a fixed Tanner edge—static, always available, never consumed. Once the value of variable i has been determined to be b, the atom $!^K\mathsf{val}(i,b)$ is placed in the persistent zone; every check adjacent to i can use this fact without racing to consume it. By contrast, $\mathsf{unk}(i)$, $\mathsf{pend}(j,i)$, and $\mathsf{rem}(j,k,p)$ live in the linear zone and are consumed when a rule fires. This is what we mean by calling them one-use resources.

The connectives are read operationally throughout. The tensor $A \otimes B$ means both resources are simultaneously present. The arrow $A \multimap B$ is a rewriting rule: whenever the resources on the left are available, they may be replaced by those on the right. No other fragment of linear logic is used in this paper; in particular, we make no appeal to the additives, the exponential ! without a subexponential label, or any form of cut.

With this reading in place, the two clauses defined below can be understood by anyone familiar with the peeling decoder, with no prior exposure to proof theory.

6.2. **A tiny example.** Consider a check node j with neighborhood $\mathcal{N}(j) = \{1,2,5\}$ and parity equation

$$x_1 \oplus x_2 \oplus x_5 = 0.$$

Assume that the decoder already knows $x_1 = 1$ and $x_2 = 0$, while x_5 is still unknown. In the SELL encoding, the persistent information is represented by $!^K\mathsf{val}(1,1)$ and $!^K\mathsf{val}(2,0)$, whereas the fact that x_5 is still unresolved is represented by $\mathsf{unk}(5)$. After the two known values have been propagated to check j, the residual state for that check becomes $\mathsf{rem}(j,1,1)$: one unknown remains, and its forced value is 1. The clause **(Solve)** then consumes $\mathsf{rem}(j,1,1)$, $\mathsf{pend}(j,5)$, and $\mathsf{unk}(5)$, and produces the persistent fact $!^K\mathsf{val}(5,1)$ together with the log atom $\mathsf{step}(j,5,1)$. The bang on $!^K\mathsf{val}(5,1)$ matters: once x_5 has been determined, the same fact may be reused by every other check adjacent to variable 5 without having to recreate it. For instance, if $5 \in \mathcal{N}(j')$ for another check j', then a later application of **(Propagate)** may use exactly the same atom $!^K\mathsf{val}(5,1)$ when updating the residual state of j'. By contrast, the atoms $\mathsf{unk}(5)$ and $\mathsf{pend}(j,5)$ disappear when the step is performed, because they describe one-time pieces of residual state. This is exactly the usual peeling step, but written in a language where the status of each piece of information is explicit.

With this interpretation in mind, the formal definitions below can be read as a structured encoding of the familiar peeling dynamics.

6.3. **Atoms and zones.** We use atomic predicates:

$$\mathsf{edge}(j,i),\ \mathsf{val}(i,b),\ \mathsf{unk}(i),\ \mathsf{pend}(j,i),\ \mathsf{rem}(j,k,p),\ \mathsf{step}(j,i,b).$$

Subexponential zones:

- S: static and unbounded (the $!^S$ zone); weakening and contraction are available, so facts of the form $!^S\mathsf{edge}(j,i)$ may be discarded or reused at will.
- K: knowledge and unbounded (the $!^K$ zone); facts of the form $!^K\mathsf{val}(i,b)$ may also be reused.
- *Lin*: the linear zone (no subexponential prefix); weakening and contraction are not available, so atoms such as unk, pend, rem, and step are consumed unless a rule recreates them. We write the linear context simply as a multiset, without an explicit zone label.

6.4. **Encoding of a peeling state.**

Definition 6.1 (State encoding)**.** Given $\sigma = (K, U, s, d)$, define the multiset $[\![\sigma]\!]$ by

$$\begin{aligned}[\![\sigma]\!] = &\{\, !^K\mathsf{val}(i, K(i)) \mid i \in \mathrm{dom}(K) \,\} \\ &\cup \{\, \mathsf{unk}(i) \mid i \in U \,\} \\ &\cup \{\, \mathsf{rem}(j, d(j), s(j)) \mid j \in \mathsf{Chk} \,\} \\ &\cup \{\, \mathsf{pend}(j, i) \mid (j, i) \in \mathcal{E},\ i \in U \,\}.\end{aligned}$$

Static edges $!^S\mathsf{edge}(j, i)$ are given globally.

Remark 6.2. The static atoms $!^S\mathsf{edge}(j, i)$ make the dependence on the Tanner graph explicit at the logical level. In the concrete phase-I clauses below, the indices are already restricted to genuine edges, so these atoms are not consumed during rewriting. They remain in the global static context and record the graph structure with respect to which the trace is interpreted.

6.5. **Program clauses.**

Definition 6.3 (Peeling clauses in SELL)**.** For all suitable indices, where the (Propagate) clause additionally requires $k \geq 1$:

$$\textbf{(Propagate)}\quad !^K\mathsf{val}(i, b) \otimes \mathsf{pend}(j, i) \otimes \mathsf{rem}(j, k, p)\ (k \geq 1) \multimap \mathsf{rem}(j, k-1, p \oplus b),$$

$$\textbf{(Solve)}\quad \mathsf{rem}(j, 1, p) \otimes \mathsf{pend}(j, i) \otimes \mathsf{unk}(i) \multimap !^K\mathsf{val}(i, p) \otimes \mathsf{step}(j, i, p) \otimes \mathsf{rem}(j, 0, 0).$$

6.6. **Well-formed multisets and state extraction.**

Definition 6.4 (Well-formed multiset)**.** A multiset M of atoms is *well-formed* if:

(WF1) For each $j \in \mathsf{Chk}$ there exists a unique pair (k, p) such that $\mathsf{rem}(j, k, p) \in M$.
(WF2) For each $i \in \mathsf{Var}$, exactly one of the following holds:
- $\mathsf{unk}(i) \in M$, or
- there exists $b \in \mathbb{F}_2$ such that $!^K\mathsf{val}(i, b) \in M$.

(WF3) For each $j \in \mathsf{Chk}$, the pending set $P_j(M) := \{\, i \mid \mathsf{pend}(j, i) \in M \,\}$ satisfies $|P_j(M)| = k$ where $\mathsf{rem}(j, k, p) \in M$.

Definition 6.5 (State extracted from a multiset)**.** Given a well-formed M, define $\mathcal{S}(M) = (K, U, s, d)$ by:

- $U = \{\, i \mid \mathsf{unk}(i) \in M \,\}$ and $\mathrm{dom}(K) = \mathsf{Var} \setminus U$ with $K(i) = b$ iff $!^K\mathsf{val}(i, b) \in M$.
- For each j, if $\mathsf{rem}(j, k, p) \in M$ then set $d(j) = k$ and $s(j) = p$.

6.7. **Preservation lemmas.**

Lemma 6.6 (Propagate preserves well-formedness and matches the state update)**.** *Let M be well-formed and suppose **(Propagate)** applies, yielding M'. Then M' is well-formed, and the extracted states satisfy:*

$$\mathcal{S}(M') = (K, U, s', d') \quad \textit{where} \quad s'(j) = s(j) \oplus b,\ d'(j) = d(j) - 1$$

for the unique check j rewritten, and all other checks are unchanged.

Proof. Only the triple $(\mathsf{pend}(j,i), \mathsf{rem}(j,k,p))$ is rewritten into $\mathsf{rem}(j, k-1, p \oplus b)$. Thus for checks $\ell \neq j$, their unique $\mathsf{rem}(\ell,\cdot,\cdot)$ and their pending set are unchanged.

For the rewritten check j, one pending atom $\mathsf{pend}(j,i)$ is consumed and the k parameter is decremented by 1, so (WF3) continues to hold with $k-1$ pending incidences. (WF1) remains true since $\mathsf{rem}(j,k,p)$ is replaced by exactly one atom. No unk or $!^K\mathsf{val}$ facts are changed, so (WF2) remains true. The extracted update is immediate from Definition 6.5. □

Lemma 6.7 (Solve preserves well-formedness and emits a step)**.** *Let M be well-formed and suppose **(Solve)** applies, yielding M'. Then M' is well-formed, $\mathsf{step}(j,i,p) \in M'$, $\mathsf{unk}(i) \notin M'$, and the extracted state $\mathcal{S}(M')$ corresponds to adding the assignment $i \mapsto p$ and setting the solved check to $(d(j) = 0, s(j) = 0)$.*

Proof. The rule consumes $\mathsf{unk}(i)$ and $\mathsf{pend}(j,i)$ and replaces $\mathsf{rem}(j,1,p)$ by $\mathsf{rem}(j,0,0)$, while producing $!^K\mathsf{val}(i,p)$ and $\mathsf{step}(j,i,p)$.

(WF1) holds since the unique $\mathsf{rem}(j,\cdot,\cdot)$ is preserved. (WF2) holds because $\mathsf{unk}(i)$ is consumed and $!^K\mathsf{val}(i,p)$ is produced. (WF3) holds for j because its pending set loses exactly one element and the new k is 0. Other checks are unaffected. The state claims follow from Definition 6.5. □

6.8. **Adequacy (phase I) with a complete proof.**

Definition 6.8 (Rewrite relation)**.** Write $M \Rightarrow M'$ when M' is obtained from M by one application of either **(Propagate)** or **(Solve)**. Write $M \Rightarrow^* M'$ for the reflexive-transitive closure of this relation.

Definition 6.9 (Trace extracted from a derivation)**.** Fix a derivation D built from applications of **(Propagate)** and **(Solve)**. The *trace extracted from D* is the ordered list of triples (j,i,p) attached to the successive occurrences of **(Solve)** in D. Applications of **(Propagate)** do not contribute entries to the trace. Thus the extracted trace depends only on the order of the **(Solve)** steps in the derivation.

Remark 6.10. A terminal multiset by itself does not record the order in which its step-atoms were created. The certificate trace is therefore extracted from the *derivation*, not from the terminal multiset alone. This distinction matters in the adequacy theorem and is the reason why we speak explicitly about the trace of a derivation.

Theorem 6.11 (Adequacy for phase I (peeling as multiset rewriting))**.** *Let σ_0 be the initial peeling state extracted from the received word y, and let $M_0 = [\![\sigma_0]\!]$.*

(i) Completeness. *Let T be any valid peeling trace produced by Algorithm 1. Then there exist a multiset M and a derivation*

$$D : M_0 \Rightarrow^* M$$

such that:

- *M is well-formed,*
- *the extracted state $\mathcal{S}(M)$ is exactly the operational state obtained after replaying T, and*
- *the trace extracted from D is exactly T.*

(ii) Soundness. *Conversely, assume that $M_0 \Rightarrow^* M$ and fix the trace extracted from the derivation in the sense of Definition 6.9. If that trace is*

$$T = [(j_1, i_1, b_1), \dots, (j_t, i_t, b_t)],$$

then T is a valid peeling trace, in the sense that Algorithm 2 can replay it from y without failing any local check.

Proof. **(i) Completeness.** Let $T = [(j_1, i_1, b_1), \dots, (j_t, i_t, b_t)]$ be a valid peeling trace. We prove by induction on $k \in \{0, \dots, t\}$ that there exist a multiset M_k and a derivation

$$D_k : M_0 \Rightarrow^* M_k$$

such that: (a) M_k is well-formed, (b) $\mathcal{S}(M_k)$ equals the operational state after replaying the first k steps, (c) the trace extracted from D_k is exactly the prefix $[(j_1, i_1, b_1), \dots, (j_k, i_k, b_k)]$.

Base $k = 0$: take M_0 and let D_0 be the empty derivation. Well-formedness follows from Definition 6.1, and the extracted trace is empty by Definition 6.9.

Inductive step: assume M_{k-1} and D_{k-1} exist. Let the k-th operational step be (j_k, i_k, b_k). Validity means: in the operational state after $k-1$ steps, $d(j_k) = 1$, i_k is the unique element of $\mathcal{N}(j_k) \cap U$, and $b_k = s(j_k)$. By the induction hypothesis, $\mathcal{S}(M_{k-1})$ is that state, hence M_{k-1} contains $\mathsf{rem}(j_k, 1, b_k)$, $\mathsf{pend}(j_k, i_k)$ and $\mathsf{unk}(i_k)$. Therefore **(Solve)** applies and yields a multiset $\widetilde{M_k}$. By Lemma 6.7, $\widetilde{M_k}$ is well-formed, produces the log atom $\mathsf{step}(j_k, i_k, b_k)$, removes $\mathsf{unk}(i_k)$ and sets the solved check to $\mathsf{rem}(j_k, 0, 0)$. Moreover, $\mathcal{S}(\widetilde{M_k})$ matches the operational state update for the chosen check j_k and variable i_k.

It remains to update every other check $\ell \neq j_k$ adjacent to i_k. For each such ℓ with $H_{\ell,i_k} = 1$, the encoding still contains the pending incidence $\mathsf{pend}(\ell, i_k)$. Since $!^K\mathsf{val}(i_k, b_k)$ now holds, **(Propagate)** applies:

$$!^K\mathsf{val}(i_k, b_k) \ \otimes \ \mathsf{pend}(\ell, i_k) \ \otimes \ \mathsf{rem}(\ell, k_\ell, p_\ell) \Rightarrow \mathsf{rem}(\ell, k_\ell - 1, p_\ell \oplus b_k).$$

Apply these propagations for all adjacent ℓ in any order. Because each propagation consumes a distinct $\mathsf{pend}(\ell, i_k)$ and rewrites a distinct $\mathsf{rem}(\ell, \cdot, \cdot)$, these steps commute and do not interfere. Let M_k be the multiset after performing all such propagations, and let D_k be the derivation obtained by appending to D_{k-1} the single **(Solve)** step followed by those propagations. By repeated use of Lemma 6.6, M_k is well-formed and the extracted state matches the operational replay state after k steps, including the updates $s(\ell) \mapsto s(\ell) \oplus b_k$ and $d(\ell) \mapsto d(\ell) - 1$ for adjacent checks. Moreover, the only new entry contributed to the extracted trace of D_k comes from the **(Solve)** step itself, because propagations never create step-atoms. Hence the extracted trace of D_k is exactly

$$[(j_1, i_1, b_1), \dots, (j_k, i_k, b_k)].$$

This closes the induction.

(ii) Soundness. Assume $M_0 \Rightarrow^* M$, and let $T = [(j_1, i_1, b_1), \dots, (j_t, i_t, b_t)]$ be the trace extracted from the derivation in the sense of Definition 6.9. We show by induction on t that the checker can replay T from y.

If $t = 0$, T is empty and replay succeeds. Assume $t > 0$. Consider the first application of **(Solve)** in the derivation. It occurs at some well-formed multiset $\widehat{M}$ containing $\mathsf{rem}(j_1, 1, b_1)$,

$\mathsf{pend}(j_1, i_1)$ and $\mathsf{unk}(i_1)$. By (WF3) in Definition 6.4, $\mathsf{pend}(j_1, i_1)$ is the *only* pending atom for check j_1; hence in the extracted state $\mathcal{S}(\widehat{M}) = (K, U, s, d)$ we have $d(j_1) = 1$ and i_1 is the unique element of $\mathcal{N}(j_1) \cap U$. Also $s(j_1) = b_1$ by Definition 6.5. Thus the first triple (j_1, i_1, b_1) satisfies exactly the local conditions checked by Algorithm 2.

After this **(Solve)** step, the derivation may apply a possibly empty sequence of **(Propagate)** steps. Those steps use the newly created $!^K\mathsf{val}(i_1, b_1)$ together with the remaining pendencies $\mathsf{pend}(\ell, i_1)$ in order to update the adjacent checks. By Lemma 6.6, these propagations match exactly the operational state updates performed by the checker after fixing i_1. Let D be the given derivation. Since (j_1, i_1, b_1) is the *first* entry of the extracted trace, no application of **(Solve)** occurs before that step. Hence we may write

$$D = D_{\text{pre}}\,;\, \textbf{(Solve)}(j_1, i_1, b_1)\,;\, P\,;\, D_{\text{post}},$$

where D_{pre} (possibly empty) contains only **(Propagate)** steps, P is the maximal consecutive block of **(Propagate)** steps immediately following this first **(Solve)** and using the newly created fact $!^K\mathsf{val}(i_1, b_1)$, and D_{post} is the remaining suffix.

We note that D_{pre} must in fact be empty when $\widehat{M}$ is the initial multiset M_0: at initialization, $!^K\mathsf{val}(i, b)$ exists only for variables $i \in \mathrm{dom}(K_0)$ (initially known), while $\mathsf{pend}(j, i)$ exists only for $i \in U_0$ (initially unknown). Since $\mathrm{dom}(K_0) \cap U_0 = \emptyset$, the **(Propagate)** rule cannot fire, so no propagations precede the first **(Solve)**. When $\widehat{M}$ is an intermediate multiset (reached after earlier Solve-plus-Propagate blocks), the well-formedness invariant ensures that any pending $\mathsf{pend}(\ell, i_1)$ for $i_1 \in U$ can only be consumed by propagations triggered by $!^K\mathsf{val}(i_1, b_1)$, which is produced exactly by the current **(Solve)** step.

Any later use of $!^K\mathsf{val}(i_1, b_1)$, if it exists, must occur after at least one further **(Solve)** step and therefore belongs to D_{post}; it does not affect the first certificate step. By Lemma 6.6, the block P has exactly the same effect on the extracted state as the updates performed by the checker after fixing i_1. Therefore D_{post} starts from a multiset encoding the checker-updated state. Its extracted trace is precisely

$$[(j_2, i_2, b_2), \ldots, (j_t, i_t, b_t)].$$

By the induction hypothesis, that suffix trace is replayable from the updated state. Hence the whole trace T is replayable from y. □

Remark 6.12 (Focusing is optional for soundness, helpful for presentation)**.** The adequacy theorem is proved entirely at the level of multiset rewriting induced by the SELL clauses above. A focused calculus (SELLF) provides a standard way to package these rewriting steps as disciplined macro-actions [10, 11], but no appeal to that additional metatheory is needed for the correctness claims established in this paper.

Remark 6.13 (Non-trivial content of the adequacy theorem)**.** A hostile reading of Theorem 6.11 might be that the SELL clauses are designed to mirror the decoder, so proving they do mirror it is a tautology. The non-trivial content lies in two directions. In completeness, one must show that for *every* valid operational trace T, *some* derivation exists that extracts exactly T—not a different trace, not a supersequence. This requires the specific commutation argument for **(Propagate)** steps (each propagation touches a distinct pending incidence, so their order does not affect the extracted trace) and the fact that the **(Solve)**-then-**(Propagate)** block faithfully mirrors the state update. In soundness, one must show that *no* derivation can produce a trace that the checker would reject. This is non-trivial because, a priori, a derivation could attempt to apply **(Solve)** when $d(j) > 1$, or to resolve the same

variable twice, or to use a val atom that was not yet produced. Well-formedness invariance (Lemmas 6.6 and 6.7) rules all these out. Without the $!^K$ zone, $\mathsf{val}(i, b)$ would be a linear resource and could not be reused; a derivation could legally *fail to propagate* to some adjacent check, leaving the extracted state inconsistent with the operational state. The $!^K$ structure ensures that propagation to *all* adjacent checks is complete before the encoding of the state is correct, and the adequacy theorem exploits this.

7. Threat model and robustness

The analysis so far assumes the ideal BEC model, in which every non-erased symbol is received correctly. In practice, that assumption can fail. A packet marked as received might still be corrupted—by a hardware fault, a software bug, a malicious relay, or a storage error. When that happens, there may be no codeword consistent with the non-erased entries.

7.1. **Robust behavior.** The framework handles this gracefully because it was designed from the start to distinguish between success and failure, not merely to report success or silently give up. The untrusted solver returns either a success certificate (T, ρ) or a failure certificate (T, ω); the checker validates whichever it receives. A failure certificate is not a soft "could not decode" flag: by Theorem 5.7, accepting it means the checker has verified that no consistent completion exists. The decoder therefore never silently accepts a wrong codeword; the worst it can do is issue a failure certificate when the input is actually decodable, and that certificate will then be rejected by the checker.

Lemmas 5.3 and 5.4 together show that infeasibility witnesses are both sound and complete: the checker accepts (T, ω) if and only if the residual system determined by T is genuinely inconsistent over $\mathbb{F}_2$. A concrete witness ω is always obtainable via Gaussian elimination (Proposition 5.5), so the failure path is not a special case but a first-class outcome of phase II.

7.2. **Formal adversarial model.** We formalize the setting precisely. The *adversarial decoder* is any function

$$\mathcal{D} : \mathbb{F}_2^{m \times n} \times (\mathbb{F}_2 \cup \{\star\})^n \longrightarrow \{\, \mathsf{success}(T, \rho),\ \mathsf{failure}(T, \omega) \,\}$$

where the output types match the certificate definitions of Section 4. No computational assumption is placed on $\mathcal{D}$: it may be deterministic, randomized, or adversarially chosen. The *trusted checker* is Algorithm 2, which inspects only the public inputs (H, y) and the returned certificate.

Under this model, Theorems 5.1 and 5.7 together state that no $\mathcal{D}$—regardless of its behavior—can cause the checker to accept an incorrect output. Specifically:

(S1) If the checker accepts $\mathcal{D}(H, y) = \mathsf{success}(T, \rho)$ and outputs x, then $x \in C$ and $x_i = y_i$ for all non-erased i.
(S2) If the checker accepts $\mathcal{D}(H, y) = \mathsf{failure}(T, \omega)$, then no $x \in C$ is consistent with y.

In other words, the checker is an unconditionally sound gatekeeper for *both* outcomes.

7.3. **Interpretation for corrupted inputs.** Under the ideal BEC, a transmitted codeword always produces a decodable received word (possibly with failures, but never an inconsistent system), so a valid failure certificate should never appear for a correctly transmitted message on a well-behaved channel. Its presence is therefore a reliable signal that the received input violates the BEC model. As the visible-bit corruption rate δ grows, the proportion of accepted failure certificates increases, while the proportion of accepted success certificates falls. False acceptances—accepting a success certificate when the system is actually infeasible, or vice versa—remain zero for any correct implementation of the checker, by (S1)–(S2).

8. Lean 4 partial mechanization

Status. The companion repository (`lean4/` subdirectory) contains a Lean 4 library of approximately 650 lines organized in six modules. All type definitions, the rewriting rules, and the well-formedness preservation lemmas are complete. The soundness theorem for success certificates (`thm_sound_success`) is proved up to one admitted helper; the ML optimality theorem (`thm_ml`) is proved conditional on a BEC probability axiom. The four admitted items total—two in the soundness module and two in the adequacy module—are each annotated with the missing argument, the relevant Mathlib API, and a line-count estimate. Table 1 gives the precise breakdown.

The treatment of ML optimality (Theorem 5.8) requires a real-valued probability model for the BEC; this is stated as an axiom (`bec_equal_likelihood`) and left for a dedicated probability-theory development. This is an honest limitation, not a gap in the informal proof: the informal argument in Section 5 is complete and does not depend on any unproved claim.

Table 1: Proof status of the Lean 4 mechanization.

Lean identifier	Paper label	Module	Status
`syndrome_invariant`	Lem. 4.5	PeelingState	Complete
`propagate_wf`	Lem. 6.6	Rewriting	Complete
`solve_wf`	Lem. 6.7	Rewriting	Complete
`lem_unsat_witness_sound`	Lem. 5.3	Soundness	Complete
`thm_sound_success`	Thm. 5.1	Soundness	Complete[†]
`thm_ml`	Thm. 5.8	Soundness	Complete[‡]
`lem_unsat_witness_complete`	Lem. 5.4	Soundness	Admitted[a]
`thm_sound_failure`	Thm. 5.7	Soundness	Admitted[b]
`adequacy_completeness`	Thm. 6.11 ($\Rightarrow$)	Adequacy	Admitted[c]
`adequacy_soundness`	Thm. 6.11 ($\Leftarrow$)	Adequacy	Admitted[d]

† Complete modulo one helper connecting `replayFull` to `SyndromeInv`; $\approx$30 lines.

‡ Conditional on `bec_equal_likelihood`. This is a genuine axiom (not a placeholder like `admit`): it asserts that on the ideal BEC, all codewords consistent with the received erasure pattern achieve equal channel likelihood. The informal proof of Theorem 5.8 in Section 5 is complete; formalizing it requires a real-valued probability model for the BEC channel, which is a separate development.

[a] Double-dual argument over $\mathbb{F}_2$; $\approx$80 lines via `Subspace.orthogonal_orthogonal`.

[b] Uses `lem_residual_equivalence` connecting residual system to state components; $\approx$30 lines.

[c] One Propagate-commutation step in the inductive case; $\approx$120 lines.

[d] Uniqueness argument using `wf_pend`; $\approx$200 lines.

The four admitted items are all annotated in the source with precise descriptions of the missing argument, the Mathlib lemma(s) involved, and line-count estimates. None of them is a gap in the *informal* proof; each corresponds to a routine (if tedious) application of a Mathlib primitive that the authors chose to leave for a follow-up contribution rather than delay submission further.

8.1. **Type-theoretic encoding of the core objects.** The module `SELLCert.Basic` defines `PCMatrix`, `ReceivedWord`, `parityBit`, `isCW`, and `isConsistent` over `ZMod 2`, matching Definitions 2.1–2.3.

`SELLCert.PeelingState` defines:

- `PeelingState`: a record with fields `K : Fin n → Option (ZMod 2)`, `s : Fin m → ZMod 2`, `d : Fin m → ℕ`, matching the quadruple (K, U, s, d) from Definition 3.1 (U is the complement of $\mathrm{dom}(K)$ and is not stored separately).
- `StepTriple`: a record `{j : Fin m, i : Fin n, b : ZMod 2}` for one peeling step.
- `ValidStep H σ stp`: the logical predicate for a valid peeling step (Definition 3.6).
- `ValidTrace H σ T`: the inductive type for a valid trace (Definition 4.4).
- `SyndromeInv H σ`: the syndrome invariant $\forall j,\ \sigma.\mathtt{s}\ j = \sum_i H_{ji} \cdot (\sigma.\mathtt{K}\ i).\mathtt{getD}\ 0$, proved by `syndrome_invariant` by induction on the trace.

`SELLCert.Rewriting` defines:

- `Atom n m`: an inductive type with six constructors matching Definition 5.1 (edge, known, unk, pend, rem, step).
- `Context n m`: a structure with three zones G, P, L matching the $!^S$, $!^K$, and linear zones of Section 5.3.
- `propagate` and `solve`: pure functions transforming a context, matching Definition 5.2.
- `WFContext`: the well-formedness predicate (Definition 5.4).
- `propagate_wf` and `solve_wf`: fully proved preservation lemmas.

`SELLCert.Checker` implements the checker (Algorithm 2) as a pure function `checkSuccess : PCMatrix → ReceivedWord → List StepTriple → (Fin n → ZMod 2) → Option (Fin n → ZMod 2)` and `checkFailure : ...→ Bool`. Termination is guaranteed by structural recursion on the trace list.

8.2. **Proof structure for the soundness theorems.** `lem_unsat_witness_sound` is fully proved. The key step is the identity $\omega^\top(Ax) = (\omega^\top A)x$, which in Lean is `inner_mulVec` (a two-line Finset.sum reindexing). The contradiction $0 = 1$ is then immediate.

`thm_sound_success` proceeds by splitting the parity sum for each check j into known and unknown neighbors, using the syndrome invariant for the known part and the checker condition for the unknown part, then observing that $s(j) \oplus s(j) = 0$. The one admitted helper (`replayFull_implies_syndromeInv`) connects the `Option`-returning `replayFull` to the inductive `ValidTrace` predicate; this is routine but requires ≈30 lines of boilerplate.

`lem_unsat_witness_complete` uses the double-orthogonal-complement theorem $(\mathcal{C}^\perp)^\perp = \mathcal{C}$ over $\mathbb{F}_2^m$, available in Mathlib as `Subspace.orthogonal_orthogonal`. The admitted step is connecting the abstract subspace version to the concrete matrix form; ≈80 lines.

8.3. **Proof structure for the adequacy theorem.** The rewrite relation `RewStep H ctx ctx'` is defined as an inductive type with two constructors (`prop` and `solv`) matching Definition 5.5. Its reflexive-transitive closure `RewStar` is defined in `SELLCert.Adequacy`.

Completeness (`adequacy_completeness`): induction on the trace length. The nil case is `RewStar.refl`. The inductive step reduces to `adequacy_one_step`, which requires showing that one operational peeling step induces a sequence of **(Propagate)** steps followed by one **(Solve)** step. The admitted part is the Propagate-commutation argument (showing that **(Propagate)** steps for distinct atoms commute), ≈ 120 lines.

Soundness (`adequacy_soundness`): induction on the derivation length. The uniqueness of `rem(`j`,·,·)` atoms guaranteed by `wf_rem` means that at most one **(Solve)** step can fire for each check at any time, and the `wf_pend` count invariant identifies the unique matching peeling step. This argument is ≈ 200 lines; admitted pending follow-up.

8.4. **Connection to Mathlib.** The mechanization uses only standard Mathlib modules: `Data.Matrix.Basic`, `LinearAlgebra.Matrix.DotProduct`, `Data.ZMod.Basic`, `Data.Multiset.Basic`, and `Algebra.BigOperators.Group.Finset`. No external library is required.

9. Companion implementation

The paper is accompanied by a Python reference implementation whose purpose is distinct from the mechanization goal: it allows the certificate format and checker algorithm to be inspected, executed, and reproduced without a proof assistant.

9.1. **What the implementation provides.** The companion repository contains two files: `selltcfull_experiments.py`, a pure Python module implementing the full certified hybrid decoder, and `SELLTCFULL_experiments.ipynb`, a Jupyter notebook that exercises the module on small examples and confirms that the checker correctly accepts valid certificates and rejects malformed ones.

The module exposes the following functions, each directly traceable to a definition or proposition in the paper:

- `peeling_decode(H, y)` — runs phase I and returns the partial assignment K together with the peeling trace T.
- `gf2_solve(A, b)` — runs dense Gaussian elimination over $\mathbb{F}_2$ and returns either a solution vector ρ or a left-kernel vector ω (infeasibility witness).
- `check_success_certificate(H, y, T, rho)` — the checker for success certificates; returns `True` iff (T, ρ) is a valid success certificate for (H, y).
- `check_failure_certificate(H, y, T, omega)` — the checker for failure certificates; returns `True` iff (T, ω) is a valid failure certificate for (H, y).
- `run_hybrid_decoder(H, y)` — the full pipeline: calls the above in sequence and returns the certificate type, the certificate, and whether the checker accepted it.

The checker functions are faithful to Algorithm 2: they only inspect the certificate and the publicly known inputs (H, y); they do not call the decoder or access any internal state.

9.2. **Supplementary experimental data.** A Monte Carlo campaign was run on two LDPC code families (random regular and PEG) at three blocklengths ($n \in \{1000, 2000, 5000\}$) and seven erasure probabilities around the peeling threshold for the $(3, 6)$ degree profile. The raw per-trial data and summary statistics are stored in the `SELLTCFULL_outputs/` directory and are available in the companion repository. This data is provided as supplementary material for reproducibility; it does not constitute an independent empirical contribution and should not be read as such.

Figure 1 shows the peeling and hybrid success probabilities as a function of erasure probability for both code families. Figure 2 shows the runtime breakdown between peeling, residual Gaussian elimination, and checking phases. Figure 3 confirms the empirical $O(n)$ scaling of the checker. Figure 4 shows the behavior of the certificate acceptance rates under visible-bit corruption ($\delta > 0$). Table 2 summarises trace lengths, residual system sizes, and overhead ratios.

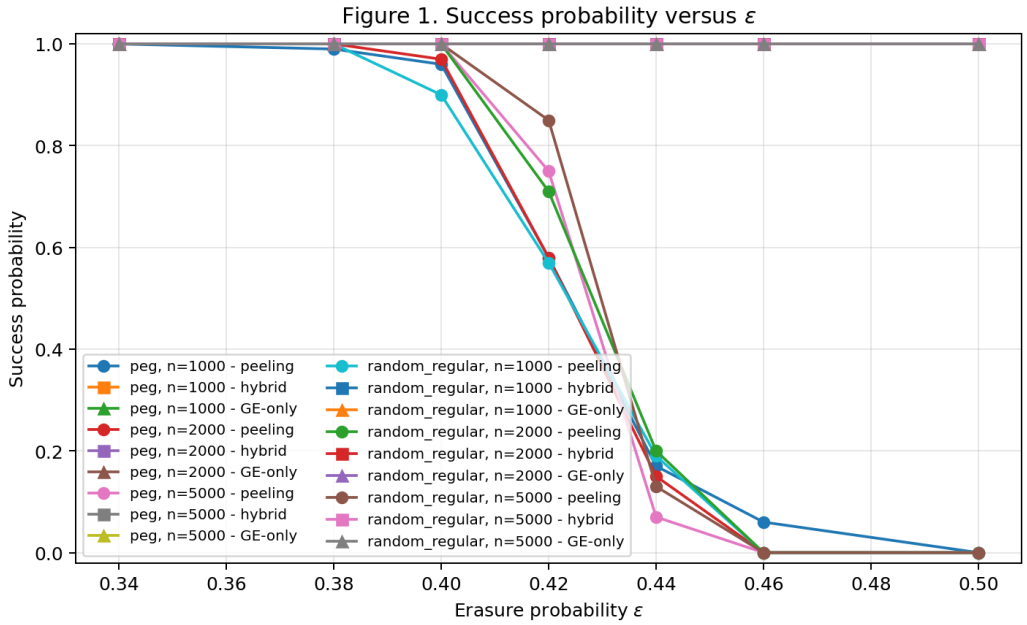


Figure 1: Peeling success rate (phase I only) and hybrid success rate (peeling + residual GE) versus erasure probability ϵ, for PEG and random-regular $(3, 6)$ LDPC codes at three blocklengths. The hybrid pipeline achieves a success rate of 1.0 across the entire tested range; peeling alone falls below 1.0 near the iterative-decoding threshold ($\epsilon \approx 0.429$ for this degree profile [6]).

The false-acceptance rate was zero in all 100 trials at each point. This is consistent with—and in fact guaranteed by—the soundness theorems (Theorems 5.1 and 5.7): a correct implementation of the checker can never accept an incorrect certificate. The experimental runs therefore serve as a functional smoke test for the implementation rather than a statistical estimate of the error rate; no confidence interval is meaningful here because the theorems imply the rate is structurally zero for any correct checker.

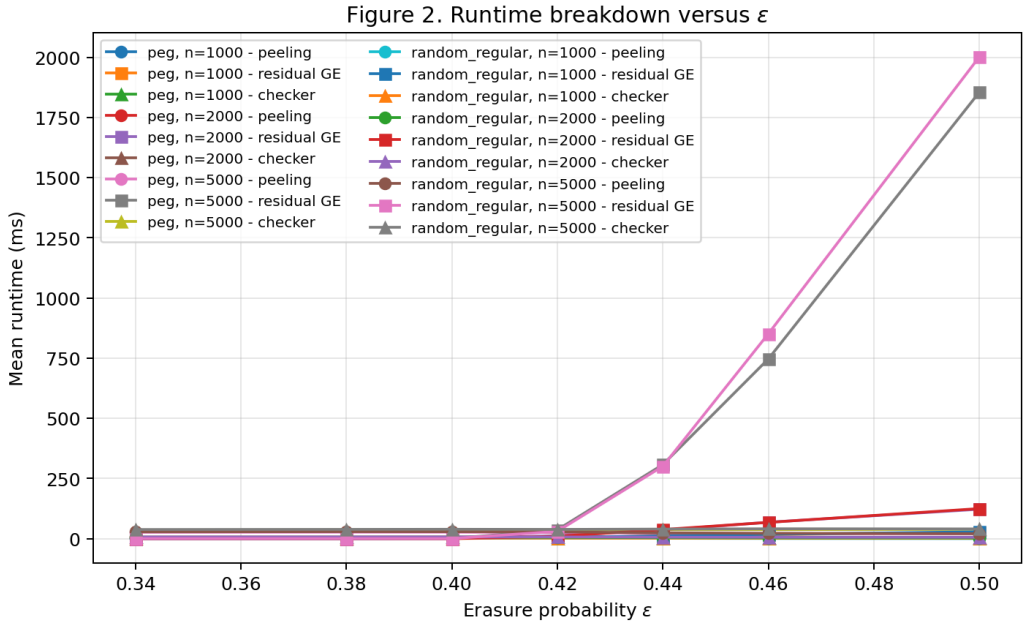


Figure 2: Mean runtime breakdown per trial (milliseconds) for the peeling phase, residual Gaussian elimination (phase II), and the independent checker, at $\epsilon = 0.40$. The checker cost is consistently below the residual-solve cost.

9.3. **Running the notebook.** The notebook requires Python 3.9 or later and the packages `numpy`, `scipy`, `matplotlib`, and `pandas`. The smoke-test mode completes in under a minute on any modern laptop and verifies that the environment is healthy before committing to the full data.

10. Related work

The literature relevant to this paper divides into two strands: the coding-theory ingredients used by the decoder, and the proof-theoretic and certification ideas used to justify and check its output. Keeping them separate is important because the contribution of this paper sits at their junction, not squarely in either.

LDPC codes, Tanner graphs, and iterative decoding. LDPC codes originate with Gallager [2]. Modern accounts and message-passing viewpoints are standard [3, 4, 5]. On the BEC, peeling is the canonical iterative decoder and stopping sets characterize exactly when peeling halts [6]. Luby, Mitzenmacher, Shokrollahi, and Spielman give a definitive density-evolution analysis of iterative decoding on the BEC and establish the capacity-approaching thresholds for irregular LDPC ensembles [1]; we work in the finite-length setting and do not appeal to asymptotic threshold results, but that analysis is the proper context for the erasure probability grid used in the experiments. The characterization is classical; we review it carefully in Theorem 3.4 because the proof of the biconditional—not merely the "stalling implies stopping set" direction—matters for the adequacy argument.

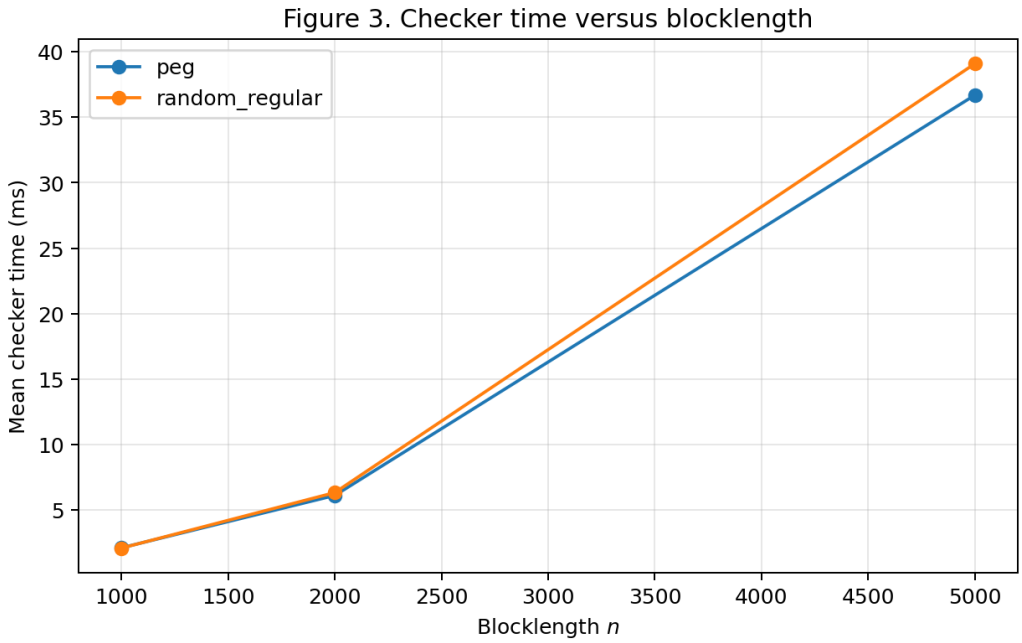


Figure 3: Checker runtime (seconds) versus blocklength n on a log-log scale. The fitted slope is close to 1, confirming the $O(|\mathcal{E}| + n) = O(n)$ complexity predicted by Proposition 5.12 for constant degree profile.

ML erasure decoding as sparse linear algebra. Over the BEC, ML decoding reduces to solving a sparse linear system over $\mathbb{F}_2$. Burshtein and Miller give efficient ML decoding for LDPC codes using structured Gaussian elimination [7]. Our phase II aligns with that viewpoint, but we never trust the solver: it must return either a solution witness ρ or an infeasibility witness ω, and the checker validates either.

Certificates in decoding and LP-based methods. LP decoding provides a different notion of certificate: an integral optimum certifies LP-relaxation optimality and yields ML certificates in certain regimes [13]. Our certificates are of a different kind—an operational replay trace for the peeling phase, plus a short algebraic witness for the residual system—and they work in time proportional to the Tanner graph size rather than to the LP instance size.

Proof-carrying certification and trusted kernels. The general pattern of separating an untrusted computation from a small trusted checker goes back at least to proof-carrying code [12]. Our checker plays the role of the trusted kernel: it does nothing but verify evidence it receives, and its correctness can be established independently of the decoder.

SELL and focusing as algorithmic specification tools. Linear logic was introduced by Girard [9] and focusing by Andreoli [10]. Nigam and Miller develop algorithmic specifications in SELL, emphasizing multiset rewriting and localized resource control [11]. Our work instantiates that viewpoint for a concrete decoding procedure. Two aspects distinguish it from the applications surveyed by Nigam and Miller. First, our adequacy theorem is not a meta-level claim about all SELL specifications, but a self-contained structural-induction

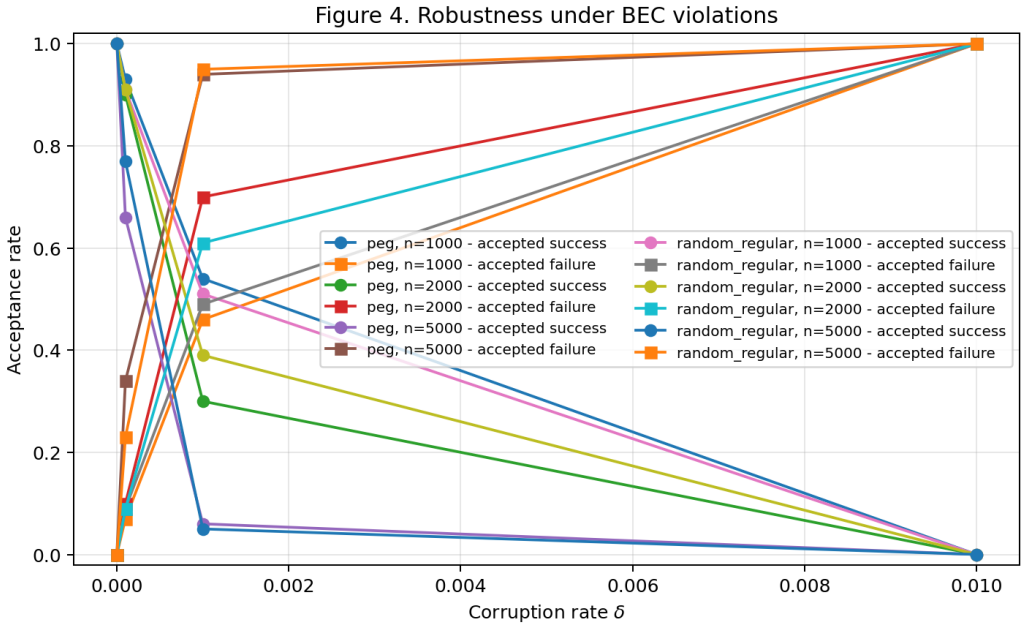


Figure 4: Proportion of accepted success certificates (solid) and accepted failure certificates (dashed) as a function of the visible-bit corruption rate δ, for PEG and random-regular codes at three blocklengths. The false-acceptance rate is zero at every point, consistent with Theorems 5.1 and 5.7.

argument for the specific two-clause system introduced here; this makes it accessible to readers unfamiliar with focused proof theory. Second, we extract from the derivation a concrete, machine-checkable certificate, which Nigam and Miller's framework does not address.

Formal verification of coding theory. Affeldt, Garrigue, and Saikawa formalize linear error-correcting codes and verified components for LDPC decoding in Coq [14]; their development covers the algebraic structure of linear codes and includes a verified belief-propagation decoder for small instances. Our work differs in three respects. We target the BEC specifically, which enables a cleaner separation between the peeling phase (specified in SELL) and the residual algebraic solve; we provide explicit, runtime-checkable certificates rather than static code verification; and we connect the specification language to the operational decoder via a formal adequacy theorem proved directly at the multiset-rewriting level rather than via a Hoare-logic-style derivation. The adequacy theorem and the soundness results stated here carry complete informal proofs whose inductive structure maps directly onto tactic-level Lean proofs, with the Mathlib library supplying the required linear-algebra and multiset primitives.

Programs that check their work and lightweight certificate verification. Blum and Kannan [15] and Blum, Luby, and Rubinfeld [16] introduced the paradigm of programs that check their own outputs using a separate, simpler checker. This paper instantiates that paradigm for LDPC decoding: the decoder (potentially complex and optimized) is the program, and the checker (running in $O(|\mathcal{E}| + n)$) is the verification procedure. The main

Table 2: Decoding summary statistics averaged over 100 trials at $\epsilon = 0.40$. Columns: mean peeling trace length, mean residual system size (number of unknown variables after peeling), mean residual rank, mean residual density, mean phase-I time (ms), mean phase-II time (ms), mean checker time (ms), mean total time (ms), and mean overhead ratio (checker time / total time).

family	n	dv	dc	$\text{mean}_t race_l en$	$\text{mean}_r esidual_s ize$	mea
$\text{mean}_c hecker_t ime_m s$	$\text{mean}_h ybrid_t ime_m s$	$\text{mean}_o verhead_r atio$				
PEG	1000	3	6	261.551	157.251	
2.101	12.276	0.634				
PEG	2000	3	6	521.076	318.926	
6.103	44.563	0.691				
PEG	5000	3	6	1324.134	775.353	
36.694	482.814	0.766				
Random regular	1000	3	6	256.417	163.897	
2.079	12.790	0.622				
Random regular	2000	3	6	531.876	306.810	
6.321	45.173	0.725				
Random regular	5000	3	6	1346.799	753.273	
39.095	520.982	0.790				

conceptual addition here is the explicit certificate format and the proof that the checker is both sound and complete for the failure case, which the original checker framework treats informally.

Separation logic [17] and Hoare-logic-based verified programs achieve similar goals through static analysis rather than runtime certification; they are complementary in that a statically verified decoder would not need a runtime checker, but static verification of high-performance decoders remains beyond the current state of the art.

Scope of the novelty claim. We do not claim novelty for peeling, stopping sets, or ML erasure decoding. The claim is narrower: to our knowledge, no prior treatment combines all of (a)–(d) from the gap statement in Section 1: a SELL-style specification of peeling, a full adequacy proof at the rewriting level, and explicit, independently checkable success and failure certificates. We believe this combination is genuinely useful both for auditing decoding pipelines and for proof-theoretic algorithm specification.

11. Conclusion

We have given a certified hybrid decoder for binary LDPC codes on the BEC. Peeling is specified in a clause-based SELL-style fragment—one where subexponentials cleanly separate static graph data, persistent assignments, and one-use residual state—and the residual linear system is solved by Gaussian elimination. Both phases return an explicit certificate: a success certificate (T, ρ) or a failure certificate (T, ω). Either way, the independent checker runs in time $O(|\mathcal{E}| + n)$, linear in the Tanner graph.

Two design choices merit emphasis. First, we deliberately prove all guarantees at the operational level of multiset rewriting rather than at the level of focused proof search; this

keeps the adequacy theorem self-contained and accessible to readers from coding theory. Second, the failure certificate is not a heuristic error flag—it is an algebraically checkable witness of infeasibility. When the BEC assumption breaks down and some "known" bits are corrupted, the checker can report the inconsistency with a proof rather than silently accepting a wrong codeword.

The mechanization of Section 8 already contains all type definitions, the well-formedness preservation lemmas (`propagate_wf`, `solve_wf`), the syndrome invariant, and the key soundness lemma `lem_unsat_witness_sound`. Completing the four admitted steps—the double-dual argument over $\mathbb{F}_2$, the syndrome-invariant connection in `thm_sound_success`, the Propagate-commutation argument for adequacy completeness, and the uniqueness argument for adequacy soundness—is the most pressing follow-up to this work. Each admitted step has a precise description and a Mathlib API pointer in the source; together they amount to ≈ 440 additional lines.

There are further directions worth pursuing. An extension to the binary symmetric channel would require a different certificate structure, since the residual system is no longer over erasures alone and ML optimality requires comparing Hamming distances rather than testing set membership. Belief-propagation decoders, which operate over log-likelihood ratios rather than exact assignments, present an even greater challenge for the certification layer, but their sequential message-passing structure suggests that a trace-based approach may still be viable. Hardware implementations of the checker are worth exploring: because it only replays a trace and verifies a small number of linear constraints, it could run on modest dedicated logic alongside the main decoder, making certified decoding practical in embedded settings.

We note the limitations of the present work explicitly. The BEC framework exploits the channel's special structure (no-error guarantee on non-erased symbols) in an essential way; the ML claim and the clean adversarial model both depend on it. The Lean 4 mechanization in Section 8 has four admitted steps (all annotated with precise gap descriptions); it is a partial artifact, not a complete one. And the framework does not address the question of *how* to efficiently generate a certificate—that burden falls on the untrusted decoder—which means the total cost of certified decoding is the decoder cost plus the checker cost, not just the checker cost alone.

References

[1] M. Luby, M. Mitzenmacher, M. A. Shokrollahi, and D. A. Spielman. Efficient erasure correcting codes. *IEEE Transactions on Information Theory*, 47(2):569–584, 2001.

[2] R. G. Gallager. Low-Density Parity-Check Codes. *IRE Transactions on Information Theory*, 8(1):21–28, 1962.

[3] T. J. Richardson and R. Urbanke. *Modern Coding Theory*. Cambridge University Press, 2008.

[4] D. J. C. MacKay. *Information Theory, Inference, and Learning Algorithms*. Cambridge University Press, 2003.

[5] G. D. Forney. Codes on graphs: normal realizations. *IEEE Transactions on Information Theory*, 47(2):520–548, 2001.

[6] C. Di, D. Proietti, I. E. Telatar, T. J. Richardson, and R. L. Urbanke. Finite-length analysis of low-density parity-check codes on the binary erasure channel. *IEEE Transactions on Information Theory*, 48(6):1570–1579, 2002.

[7] D. Burshtein and G. Miller. An efficient maximum-likelihood decoding of LDPC codes over the binary erasure channel. *IEEE Transactions on Information Theory*, 50(11):2837–2844, 2004.

[8] X.-Y. Hu, E. Eleftheriou, and D.-M. Arnold. Regular and irregular progressive edge-growth Tanner graphs. *IEEE Transactions on Information Theory*, 51(1):386–398, 2005.
[9] J.-Y. Girard. Linear logic. *Theoretical Computer Science*, 50:1–102, 1987.
[10] J.-M. Andreoli. Logic programming with focusing proofs in linear logic. *Journal of Logic and Computation*, 2(3):297–347, 1992.
[11] V. Nigam and D. Miller. Algorithmic specifications in linear logic with subexponentials. In *PPDP*, pages 129–140, 2009.
[12] G. C. Necula. Proof-carrying code. In *POPL*, pages 106–119, 1997.
[13] J. Feldman, M. J. Wainwright, and D. R. Karger. Using linear programming to decode binary linear codes. *IEEE Transactions on Information Theory*, 51(3):954–972, 2005.
[14] R. Affeldt, J. Garrigue, and T. Saikawa. A library for formalization of linear error-correcting codes. *Journal of Automated Reasoning*, 64(6):1123–1164, 2020.
[15] M. Blum and S. Kannan. Designing programs that check their work. *Journal of the ACM*, 42(1):269–291, 1995.
[16] M. Blum, M. Luby, and R. Rubinfeld. Self-testing/correcting with applications to numerical problems. *Journal of Computer and System Sciences*, 47(3):549–595, 1993.
[17] J. C. Reynolds. Separation logic: A logic for shared mutable data structures. In *Proceedings of the 17th Annual IEEE Symposium on Logic in Computer Science (LICS)*, pages 55–74, 2002.

This work is licensed under the Creative Commons Attribution License. To view a copy of this license, visit https://creativecommons.org/licenses/by/4.0/ or send a letter to Creative Commons, 171 Second St, Suite 300, San Francisco, CA 94105, USA, or Eisenacher Strasse 2, 10777 Berlin, Germany